Side-channel Analysis with Deep Learning on AES using Hamming weight Leakage

Ivor Brouwer & Ivar van der Spoel

May 2025

1 Introduction

With side-channel analysis we try to find flaws in extra information that can be gathered from an encryption algorithm, rather than flaws in the design of the protocol or algorithm itself. During the course, we attended a lecture on Side-Channel Analysis using Deep Learning (SCA-DL). This approach intrigued us, particularly the potential to combine it with the ChipWhisperer, which is a versatile tool that supports SCA such as capturing and analyzing power traces from cryptographic devices, which is the focus of this assignment.

In this assignment, our goal is to extract AES encryption keys by leveraging power consumption traces. These traces reveal patterns linked to the AES algorithm. Using a Convolutional Neural Network (CNN), we model the relationship between these traces and Hamming weight leakages to recover the secret key.

2 Background

A brief background about AES, ChipWhisperers, Hamming weights and Neural Networks with SCA is necessary.

2.1 Simple AES overview

In Advanced Encryption Standard (AES)[Sel10], the plaintext is XORed with the secret key, and the result is passed through a substitution box (S-box), which introduces non-linearity. This is visualized in Figure 1. For this assignment Simple AES is used, the secret key is 16 Bytes. Rather than this simple implementation, a standard AES algorithm performs multiple rounds of data transformations,including substitution, permutation, and mixing operations,in order to obscure the relationship between the plaintext and the ciphertext.

The AES S-box is publicly known [WT85], meaning that for any input Byte, the corresponding output is fixed and predictable. This property is crucial in side-channel attacks, as it allows an attacker to compute hypothetical intermediate values (e.g., Hamming weights) and correlate them with observed power consumption to recover key Bytes.

2.2 ChipWhisperer

The ChipWhisperer is an open-source hardware platform specifically designed for performing side-channel power analysis and fault injection attacks[OC14]. It provides many potential targets (such as AES) and a capture board that records power consumption. Through a Python API we can interact with the ChipWhisperer making it ideal for educational and research purposes in cybersecurity. An example of such a power trace can be seen in Figure 2.

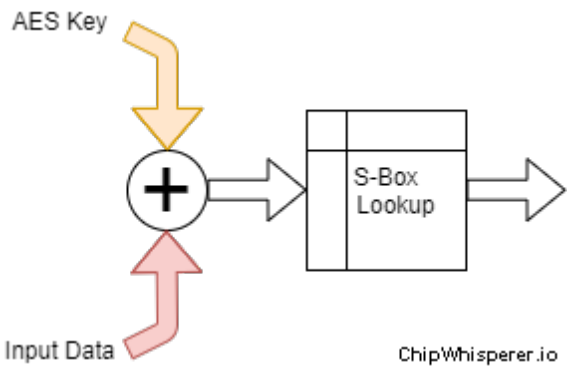


Figure 1: Visualization of a simplified AES encryption.

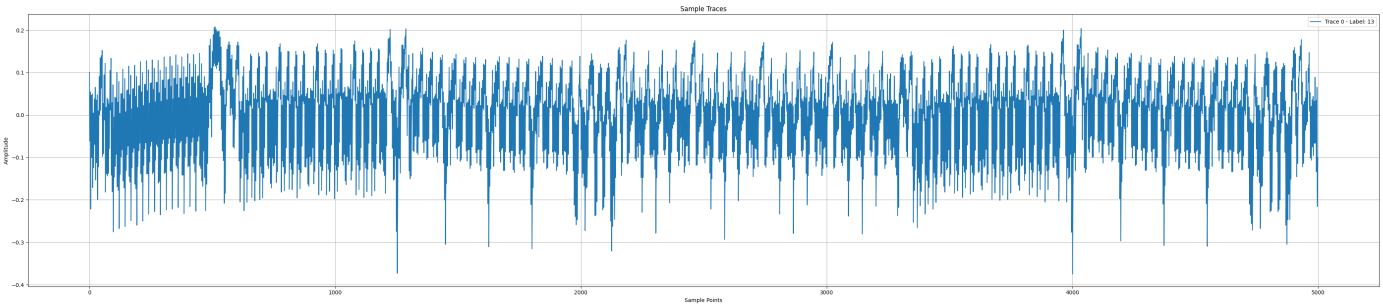


Figure 2: Example of a power trace measured by a ChipWhisperer.

2.3 Hamming weights

The Hamming weight of a Byte is the number of bits set to '1' in its binary form. In side-channel analysis, it is commonly used as a leakage model because the power consumption of digital circuits often correlates with the number of switching bits[Wei02]. During AES encryption, values such as the output of the S-box operation show power consumption patterns.

2.4 Neural Networks in Side-channel Analysis

Neural networks, particularly CNNs, are well-suited for modeling the power traces and leaked information in side-channel attacks[BPS+20]. By using a Hamming weight leakage model, we can train a neural network to classify segments of power traces according to the Hamming weight of simple AES values. This learned mapping allows the model to predict likely Hamming weight values, giving the opportunity to recover keys through statistical analysis of multiple traces.

3 Methodology

The threat model is described, the way the datasets are generated, how the CNN is built and what key guessing strategy is used.

3.1 Threat model

We assume the attacker has access to a cryptographic device performing AES encryptions with a fixed secret key. The attacker has knowledge which plaintext are being encrypted and is able to cap-

ture the corresponding power consumption traces. The attacker knows the AES algorithm (Simple AES). Alongside this, the attacker has a trained Neural Network which can sufficiently predict the Hamming weight of the aforementioned encryption algorithm with the power consumption trace. This model is trained on the same algorithm with variable keys.

However, the attacker does not know the secret key and has no access to internal states of the device. The goal of the attacker is to recover the secret key by analyzing the power traces using a trained neural network model that predicts the Hamming weight of intermediate values of AES, enabling guesses of key Bytes to be ranked and eventually determined.

3.2 Datasets

Two different datasets are required: a profiling dataset and a dataset to evaluate the matching strategy. When measuring traces with a target algorithm, alongside a certain key and text to encrypt, on the ChipWhisperer, a range of 5000 timesteps is obtained.

3.2.1 Profiling dataset

To train our neural network, we use the ChipWhisperer to generate a dataset of 10,000 power traces. For each trace, a randomly chosen key is used to encrypt a plaintext input with simple AES. Alongside the recorded power trace, we store the corresponding plaintext and key. This profiling dataset is used to train the network to associate trace patterns with the Hamming weight of intermediate values.

3.2.2 Matching dataset

To evaluate the effectiveness of our trained model, we simulate an attack scenario using new, unseen data. We generate 100 traces using 100 unknown keys and encrypt corresponding known plaintexts while recording the power consumption.

3.3 Neural Network Model

To classify power traces corresponding to different AES key Bytes, we use supervised learning with labels based on the Hamming weight of the S-box output. Specifically, for each plaintext Byte and candidate key Byte, we define the label as Hamming_weight(Sbox[plaintext_Byte $\oplus$ key_Byte]) In Figure 3 an overview can be seen of the model used for training on the different key Bytes. An individual model is trained for each of the 16 key Bytes, using the same neural network. Each model takes a segment of the power trace as input and outputs a probability distribution over 9 classes (Hamming weights 0 to 8).

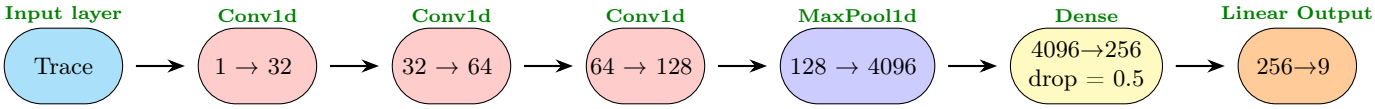


Figure 3: An overview of the CNN model used and its layers.

The goal of training is to achieve at least 50% classification accuracy on the test set, as this indicates that the model has learned meaningful correlations between the power traces and the corresponding labels. In contrast, if the model fails to exceed an accuracy of approximately 27%, this strongly suggests random guessing. Because the Hamming weight of 4 occurs in 27% of cases. Therefore, when a model achieves only 27% accuracy it is just guessing the most likely class.

Training configuration:

- **Loss function:** Cross Entropy Loss
- **Optimizer:** Adam
- **Train/test split:** 80% / 20%
- **Epochs:** 30
- **Target output:** 9-class softmax corresponding to Hamming weights

3.4 Key guessing strategy

Once the neural network models are trained to predict Hamming weights from power traces, we use their outputs to recover the AES key. For each of the 16 key Bytes, we use a separate model trained specifically for that Byte's corresponding trace segment.

During the attack phase, each model predicts the Hamming weight of the S-box output for its respective Byte across all collected traces. For each trace, we evaluate all 256 possible Byte values. Each guess is XORed with the corresponding plaintext Byte and passed through the known AES S-box. The S-box can be found in the appendix. The Hamming weight of the result is then compared to the model's prediction. If the values match, a score counter for that key Byte hypothesis is incremented.

After processing all traces, the Byte value with the highest score is selected as the predicted key Byte. This process is repeated for all 16 key Bytes, resulting in a full key guess. The entire strategy is detailed in Algorithm 1.

Algorithm 1 Key guessing

```
predicted_key ← list()                          ▷ List of predicted Bytes making up the key.
models ← 16 models trained for each Byte
sbox ← known S-box
traces ← measured traces                        ▷ n traces for key to match.
plaintext ← known plaintext                     ▷ n texts belonging to the traces.
for each Byte number b with model m in models do
    Byte_guesses ← [0, 0, 0, ...]               ▷ 256 values, for each potential key Byte value.
    for each trace t in traces do
        predicted_hw ← m(t)                     ▷ Predicted Hamming weight.
        for Byte_guess in range(256) do         ▷ Try each potential Byte.
            val ← hw(sbox[plaintext[b] ⊕ Byte_guess])   ▷ Hamming weight of guess.
            if val == predicted_hw then
                Byte_guesses[Byte_guess]++
            end if
        end for
    end for
    predicted_Byte = argmax(Byte_guesses)       ▷ Index of highest value in the list.
    predicted_key.append(predicted_Byte)
end for
```

4 Results

First we display the results of the profiling phase, and then the matching phase.

4.1 Profiling Phase Results

We found that the main leakage happens in the first 2500 sample points in the trace, when letting the models perform inference on these parts they often reach more than 50% accuracy on the test set. In Figure 4 the accuracy and loss can be seen when training the first model (performing on first Byte of the key and the first Byte of the plaintext). We then continued to train the models for the other Bytes. The ranges the models ended up using were determined through trial-and-error. The accuracy, loss and used range of each model corresponding to a single Byte can be found in Table 1.

Byte	Accuracy	Loss	Range
0	57.0%	148.864	1100-1600
1	68.15%	137.333	1300-2300
2	54.7%	143.628	1300-1800
3	69.3%	129.030	1800-2300
4	54.55%	152.634	1000-1800
5	49.65%	155.898	1200-2400
6	71.7%	130.182	1500-2500
7	67.55%	140.602	1500-2500
8	55.15%	150.329	1000-2400
9	57.9%	142.686	1800-2300
10	66.4%	134.995	1100-2500
11	57.9%	129.075	1900-2400
12	49.4%	147.512	1200-1700
13	61.45%	124.110	1900-2400
14	66.55%	139.314	1900-2400
15	49.7%	160.102	1200-2100

Table 1: Accuracy, Loss and Range per Byte Number.

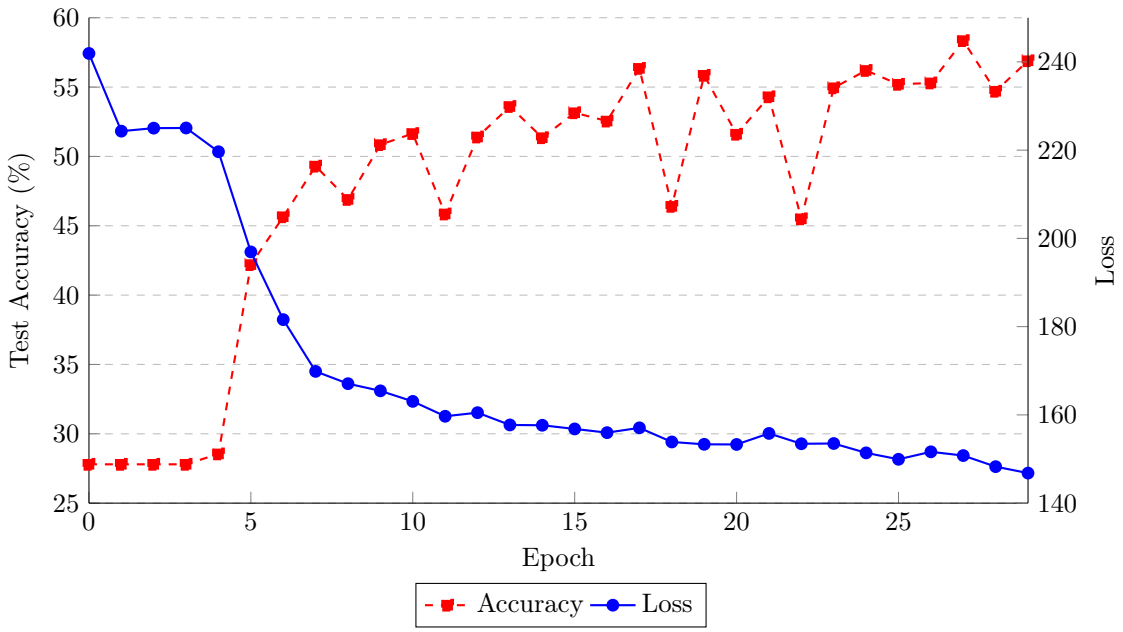


Figure 4: Test Accuracy (left axis) and Training Loss (right axis) over Epochs.

4.2 Match Phase Results

First we discuss our findings with with matching a single Byte, which we then expand to match a single key. To validate our approach, we apply the same method to recover 100 keys.

4.2.1 Recovering a single key

To recover a single Byte, we use the key guessing strategy explained in Section 3.4. We encrypt some plaintext with a fixed key in 100 traces. This results in a guessing distribution, which can be observed in Figure 5. There's a strong spike at Byte value `0x65`, which matches the correct key Byte. The red spike peaks at slightly under 60 cases, whereas the other potential key Bytes do not even reach 40 cases. This indicates the model is able to recover this Byte correctly for this case. Training each model to recover each Byte with the aforementioned models , we were able to recover the entire key. The results of each Byte of this key can be found in the Appendix.

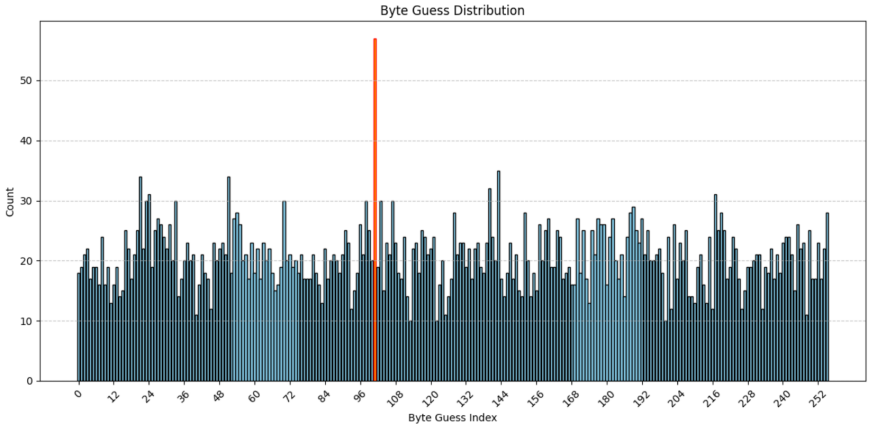


Figure 5: Guessing distribution of a Byte.

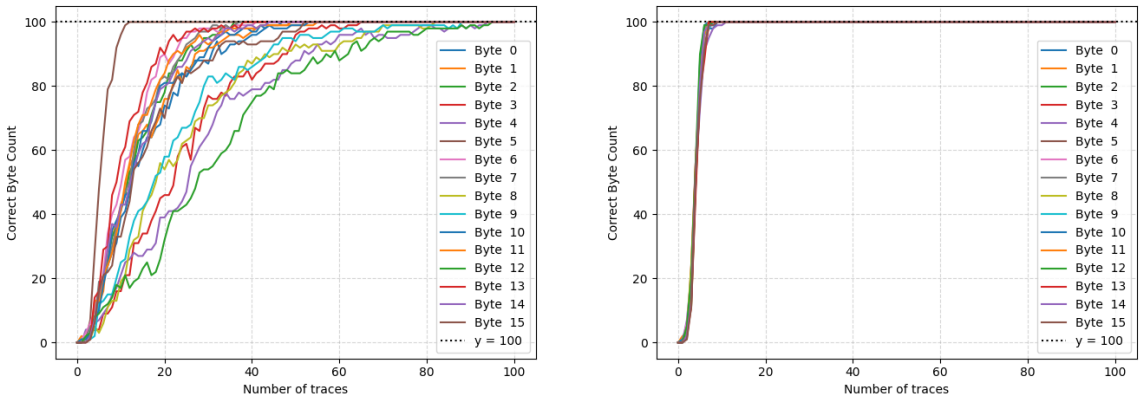
4.2.2 Recovering multiple keys

To determine the effectiveness, we match 100 keys each with 100 traces. The results can be found in Figure 6. We observe how many traces are necessary to match all Bytes for each key. There are 100 keys and 100 traces. Ideal models would be models that are 100% accurate. These models would only need at most 15 traces to recover a full key. However, as seen in the previous section our models do not match this. We are interested in how many traces are necessary to match all keys. With 100 traces, we can sufficiently predict **all Bytes for each key**. Which is a great result, because there are 256 Byte options, and we are able to filter them out with a third of the amount of traces.

Some models perform better than others, for example model 15 (corresponding to Byte 15) performs much better than the others. It cannot be said that at 40 traces we would be able to guess 75% of the keys, as one miss-matching Byte would completely invalid the key. We can say that each key is at least 75% correct.

5 Discussion

This was Simple AES, which is relatively straight forward to crack. A more difficult approach would be having AES with multiple rounds (in reality, regular AES). This has been done before [DKS10],



(a) Our models.

(b) Ideal models.

Figure 6: Comparing ideal models to our models.

but is out of scope of this project. Another countermeasure would be to implement masking in AES, this is what ASCAD (A standardized dataset for SCA-DL) does[BPS+20]. We could also perform the same attack on the ASCAD datasets. This would mean that we need to re-train each model to be able to recover the Hamming weights of ASCAD traces. Improving our models for our datasets would also be a possible extension. Instead of trial-and-error, we could use better Hyperparameter Optimization techniques such as Grid Search. In the ChipWhisperer in lab 3_3 in sca101, there are at least 2500 traces required. Even up to 10,000 to diminish ghost peaks. Whereas our models require just 100 traces.

6 Conclusion

In this assignment, we demonstrated the feasibility of side-channel attacks on AES encryption using power traces and deep learning. By training Convolutional Neural Networks on traces generated with the ChipWhisperer, we were able to recover secret AES keys by leveraging the correlation between Hamming weight leakages and power consumption.

The results show that, given access to a device performing encryption and the corresponding power traces, an attacker can recover full AES keys using only 100 traces per key. This emphasizes the importance of implementing countermeasures in cryptographic hardware to guard against side-channel vulnerabilities.

References

[BPS+20] Ryad Benadjila, Emmanuel Prouff, Rémi Strullu, Eleonora Cagli, and Cécile Dumas. Deep learning for side-channel analysis and introduction to ascad database. *Journal of Cryptographic Engineering*, 10(2):163–188, 2020.

[DKS10] Orr Dunkelman, Nathan Keller, and Adi Shamir. Improved single-key attacks on 8-round aes-192 and aes-256. In *Advances in Cryptology-ASIACRYPT 2010: 16th International Conference on the Theory and Application of Cryptology and Information Security, Singapore, December 5-9, 2010. Proceedings 16*, pages 158–176. Springer, 2010.

[OC14] Colin O'flynn and Zhizhang Chen. Chipwhisperer: An open-source platform for hardware embedded security research. In *Constructive Side-Channel Analysis and Secure Design:*

5th International Workshop, COSADE 2014, Paris, France, April 13-15, 2014. Revised Selected Papers 5, pages 243–260. Springer, 2014.

[Sel10] Douglas Selent. Advanced encryption standard. *Rivier Academic Journal*, 6(2):1–14, 2010.

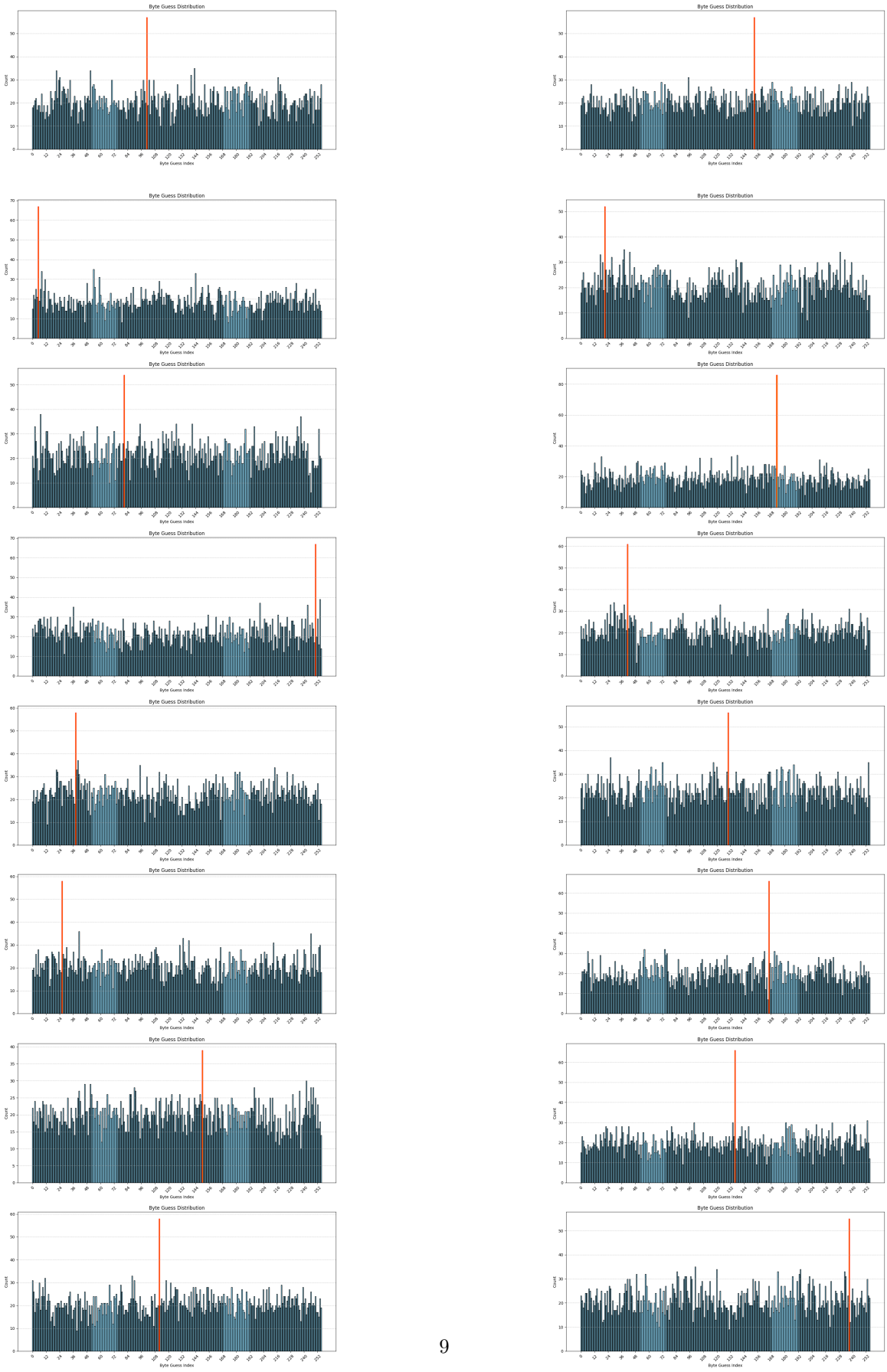
[Wei02] Victor K Wei. Generalized hamming weights for linear codes. *IEEE Transactions on information theory*, 37(5):1412–1418, 2002.

[WT85] Arthur F Webster and Stafford E Tavares. On the design of s-boxes. In *Conference on the theory and application of cryptographic techniques*, pages 523–534. Springer, 1985.

A S-box

```
    sbox = [
    # 0    1    2    3    4    5    6    7    8    9    a    b    c    d    e    f
    0x63,0x7c,0x77,0x7b,0xf2,0x6b,0x6f,0xc5,0x30,0x01,0x67,0x2b,0xfe,0xd7,0xab,0x76, # 0
    0xca,0x82,0xc9,0x7d,0xfa,0x59,0x47,0xf0,0xad,0xd4,0xa2,0xaf,0x9c,0xa4,0x72,0xc0, # 1
    0xb7,0xfd,0x93,0x26,0x36,0x3f,0xf7,0xcc,0x34,0xa5,0xe5,0xf1,0x71,0xd8,0x31,0x15, # 2
    0x04,0xc7,0x23,0xc3,0x18,0x96,0x05,0x9a,0x07,0x12,0x80,0xe2,0xeb,0x27,0xb2,0x75, # 3
    0x09,0x83,0x2c,0x1a,0x1b,0x6e,0x5a,0xa0,0x52,0x3b,0xd6,0xb3,0x29,0xe3,0x2f,0x84, # 4
    0x53,0xd1,0x00,0xed,0x20,0xfc,0xb1,0x5b,0x6a,0xcb,0xbe,0x39,0x4a,0x4c,0x58,0xcf, # 5
    0xd0,0xef,0xaa,0xfb,0x43,0x4d,0x33,0x85,0x45,0xf9,0x02,0x7f,0x50,0x3c,0x9f,0xa8, # 6
    0x51,0xa3,0x40,0x8f,0x92,0x9d,0x38,0xf5,0xbc,0xb6,0xda,0x21,0x10,0xff,0xf3,0xd2, # 7
    0xcd,0x0c,0x13,0xec,0x5f,0x97,0x44,0x17,0xc4,0xa7,0x7e,0x3d,0x64,0x5d,0x19,0x73, # 8
    0x60,0x81,0x4f,0xdc,0x22,0x2a,0x90,0x88,0x46,0xee,0xb8,0x14,0xde,0x5e,0x0b,0xdb, # 9
    0xe0,0x32,0x3a,0x0a,0x49,0x06,0x24,0x5c,0xc2,0xd3,0xac,0x62,0x91,0x95,0xe4,0x79, # a
    0xe7,0xc8,0x37,0x6d,0x8d,0xd5,0x4e,0xa9,0x6c,0x56,0xf4,0xea,0x65,0x7a,0xae,0x08, # b
    0xba,0x78,0x25,0x2e,0x1c,0xa6,0xb4,0xc6,0xe8,0xdd,0x74,0x1f,0x4b,0xbd,0x8b,0x8a, # c
    0x70,0x3e,0xb5,0x66,0x48,0x03,0xf6,0x0e,0x61,0x35,0x57,0xb9,0x86,0xc1,0x1d,0x9e, # d
    0xe1,0xf8,0x98,0x11,0x69,0xd9,0x8e,0x94,0x9b,0x1e,0x87,0xe9,0xce,0x55,0x28,0xdf, # e
    0x8c,0xa1,0x89,0x0d,0xbf,0xe6,0x42,0x68,0x41,0x99,0x2d,0x0f,0xb0,0x54,0xbb,0x16  # f
]
```

B Key guess


Byte Guess Distribution
Count
Byte Guess Index